Assignment-17.3

K . Jashwanth

2303A52300

Batch 42

Task-I: Employee Dataset Cleaning

PROMPT:

Write Python code using pandas to clean an employee dataset stored in a CSV file named "employee_data.csv". First, load the dataset and display the original data. Create a copy of the dataset for cleaning. Handle missing values by filling the salary and experience columns with their median values and replacing missing values in the designation column with "unknown". Remove duplicate records based on employee_id so each employee appears only once. Standardize text values by converting the designation column to lowercase and removing extra spaces. Display the cleaned dataset and save it as a new CSV fi le named "employee_clea ned. csv".

CODE:

Files

..

sample_data

employee_cleaned.csv

employee_data.csv

[6]

```python
import pandas as pd
# Load dataset
df = pd.read_csv("employee_data.csv")
# Show Original Data
print("Original Dataset:\n")
print(df)
# Create copy for cleaning
clean_df = df.copy()
# Fill missing values (no inplace, no warning)
clean_df['salary'] = clean_df['salary'].fillna(clean_df['salary'].median())
clean_df['experience'] = clean_df['experience'].fillna(clean_df['experience'].median())
clean_df['designation'] = clean_df['designation'].fillna('unknown')
# Remove duplicate employee IDs
clean_df = clean_df.drop_duplicates(subset='employee_id')
# Standardize text
clean_df['designation'] = clean_df['designation'].str.lower().str.strip()
# Show Cleaned Data
print("\nCleaned Dataset:\n")
print(clean_df)
# Save cleaned dataset
clean_df.to_csv("employee_cleaned.csv", index=False)
```

OUTPUT:

```
Original Dataset:

   employee_id     name   designation employment_type   salary  experience
0          101     Asha       Manager       Full-time  75000.0         8.0
1          102     Ravi  Data Analyst       Full-time  55000.0         5.0
2          103    Meena           NaN        Contract      NaN         3.0
3          104  Karthik            HR       Full-time  60000.0         NaN
4          105    Divya     Developer       Part-time  50000.0         2.0
5          106     Arun     Developer       Full-time      NaN         4.0
6          102     Ravi  Data Analyst       Full-time  55000.0         5.0
7          108    Sneha       Manager        Contract  72000.0         7.0
8          109   Vikram           NaN       Part-time  48000.0         NaN
9          110    Nisha        Intern          Intern  30000.0         1.0

Cleaned Dataset:

   employee_id     name   designation employment_type   salary  experience
0          101     Asha       manager       Full-time  75000.0         8.0
1          102     Ravi  data analyst       Full-time  55000.0         5.0
2          103    Meena       unknown        Contract  55000.0         3.0
3          104  Karthik            hr       Full-time  60000.0         4.5
4          105    Divya     developer       Part-time  50000.0         2.0
5          106     Arun     developer       Full-time  55000.0         4.0
7          108    Sneha       manager        Contract  72000.0         7.0
8          109   Vikram       unknown       Part-time  48000.0         4.5
9          110    Nisha        intern          Intern  30000.0         1.0
```

EXPLANATION:

This code cleans an employee dataset using Python's pandas library. It loads the CSV file and displays the original data for comparison. A copy of the dataset is created to safely perform cleaning operations. Missing values in salary and experience are filled with their median values, and missing designations are replaced with "unknown". Duplicate records are removed based on employee_id to ensure each employee is unique. The designation column is standardized by converting text to lowercase and removing extra spaces. Finally, the

import pandas as

import

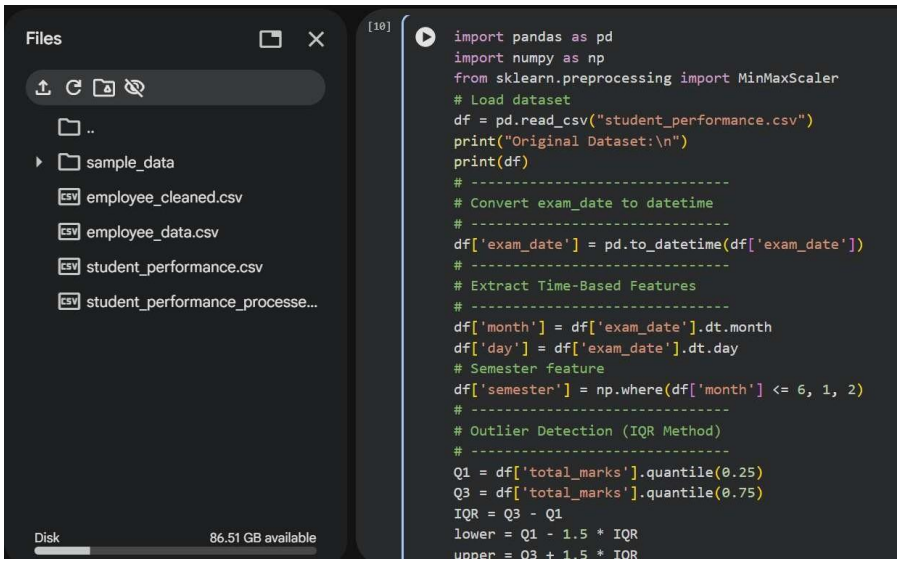
cleaned dataset is displayed and saved as a new CSV file for further use.

Task-2: Student Performance Data Preprocessing

PROMPT:

Write a Python program using Pandas, NumPy, and Scikit-learn to preprocess a student performance dataset. The program should load data from a CSV file, convert the exam date column into datetime format, and extract features such as month, day, and semester. It should detect and handle outliers in the total_marks column using the IQR method, normalize the marks using Min-Max scaling by storing them in a new column called normalized_marks, display the transformed dataset, and save it into a new CSV file.

CODE:



```
[10]
upper = Q3 + 1.5 * IQR
# Cap outliers (modify total_marks)
df['total_marks'] = np.where(df['total_marks'] < lower, lower, df['total_marks'])
df['total_marks'] = np.where(df['total_marks'] > upper, upper, df['total_marks'])
# ------------------------------
# Normalize Marks (ADD NEW COLUMN)
# ------------------------------
scaler = MinMaxScaler()
df['normalized_marks'] = scaler.fit_transform(df[['total_marks']])
# ------------------------------
# Final Output
# ------------------------------
print("\nTransformed Dataset:\n")
print(df)
# Save processed dataset
df.to_csv("student_performance_processed.csv", index=False)
```

OUTPUT:

```
Original Dataset:
         student id        name     exam
              date total marks 201 Anu
              2024-61-15
1             202 Bharat 2024-82-28         386
2             203 Charan 2024-e3-18         495
3             204 Deepa 2024-04-22    21e
4             2e5 Eshan 2024-05-3e
5             206 Farah 2024-e6-12 305
6             207 Gokul 2024-e7-25 52e
7             208 Hema 2024-e8-19  43e
8             209  Irfan 2024-69-1e       39e 9      21e        iya 2024-10-05
Transformed Dataset:

  student id name exam date total marks month day
        semester 201 Anu 2e24-01-15 450. e 1 15 1
1             202 Bharat 2e24-02-2e         380. e   2        2e
                1
2             203 Cha ran 2e24-e3-18        495.0    3        18
                1
3             204 Deepa 2e24-ø4-22 255.0    4        22       1
4             285 Eshan 2e24-e5-3e 470. e   5        3e       1
5             206 Farah 2e24-06-12 3e5.e    6        12       1
6             207 Gokul 2e24-ø7-25 520.0    7        25       2
7             208 Hema 2e24-ø8-19  430. e   8        19       2
8             209 Irfan 2e24-ø9-1e 399. e   9        2
9             21e iya 2e24-1e-e5   460.     10       5        2
```

import pandas as

import

```
...
   normalized_marks
0          0.735849
1          0.471698
2          0.905660
3          0.000000
4          0.811321
5          0.188679
6          1.000000
7          0.660377
8          0.509434
9          0.773585
```

EXPLANATION:

The program begins by importing necessary libraries such as Pandas for data handling, NumPy for numerical operations, and MinMaxScalerfor normalization. It loads the dataset from a CSV file into a DataFrame and converts the exam date column into datetime formatto enable feature extraction. From this column, it extracts the month and day, and creates a semester column based on the month value. The program then detects outliers in the total _marks column using the Interquartile Range (IQR) method by calculating QI , Q3, and IQR, and caps values that fall outside the defined lower and upper limits. After handling outliers, it normalizes the marks using Min-Max scaling, storing the scaled values in a new column called normalized_marks while keeping the original marks unchanged. Finally, it displays the processed dataset and saves it as a new CSV file.

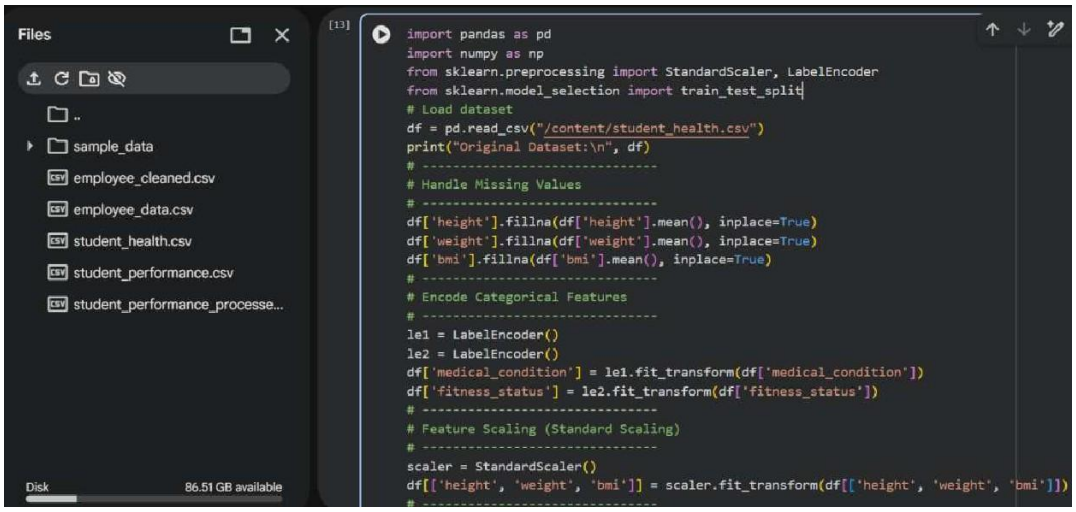
Task-3: Student Health Data Preparation

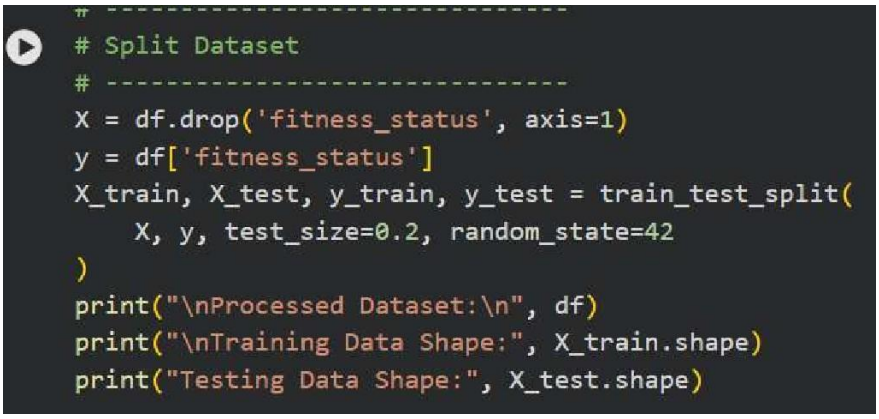
PROMPT:

Write a Python program to clean and preprocess student health records using Pandas and Scikit-learn. The program should handle missing values in numerical columns such as height, weight, and BMI, encode categorical features like medical

condition and fitness status, and scale numerical features using standard scaling. Finally, the dataset should be split into training and testing sets to prepare it for predictive health analysis.

CODE:





OUTPUT:

```
Original Dataset:
   student id  height  weight   bmi  medical condition  fitness status
        1     17e.e   65.e   22.5          NaN                Fit
1       2     165.e    NaN   24.2     Diabetes              Unfit
2       3     180. e  75.e    NaN          NaN                Fit
3       4      NaN    70.0   23.1   Hypertension          Average
```

```
                    import pandas as
                                          import

4          175.e   68 .e 22.2        NaN        Fit
5       6  168.e   72.e 25.5      Diabetes    Unfit
6       7  172.e    NaN  23.e         NaN       Fit
7       8  169.e   74. e  NaN   Hypertension  Average
8       9   NaN    69. e 24.8        NaN       Fit
9      le  178.e   80. e 25.2      Diabetes    Unfit
le     11  166. e  se.e  21.8        NaN       Fit
11     12  182.e    NaN  26.5   Hypertension  Average
12     13  174.e   77.   NaN         NaN       Fit
                    e
13     14   NaN    73.e  24.e      Diabetes    Unfit
14     15  171.e   68.e  23.3        NaN       Fit

Processed Dataset:
   student idheight   weight       bmi medical
                                   condition
          1 -e. 537086 -1. 267372 -1.084693e+ee                2
1      2 -1.611258 e. eeeeee 2.897006e-e1
2      3 1.611258 e. 874665 -2.872251e-15                      2
3          4 e. aeeeee -e. 196353 -5.996128e-e1                1
4          5 e. 537086 -e. 624761 -1.327233e+ee                2
5           6 -e. 966755 e. 232054 1.3407e7e+ee
6           7 -e.łe7417 e. eeeeee -6.804595e-e1                 2
7     8 -e. 751921 e. 660462 -2.872251e-15                      1
```

```
    7          8 -0.751921  0.660462 -2.872251e-15              1
    8          9  0.000000 -0.410557  7.747806e-01              2
...  9         10  1.181590  1.945684  1.098167e+00              0
    10         11 -1.396424 -2.338391 -1.650620e+00              2
    11         12  2.040927  0.000000  2.149174e+00              1
    12         13  0.322252  1.303073 -2.872251e-15              2
    13         14  0.000000  0.446258  1.280072e-01              0
    14         15 -0.322252 -0.624761 -4.379195e-01              2

        fitness_status
    0                1
    1                2
    2                1
    3                0
    4                1
    5                2
    6                1
    7                0
    8                1
    9                2
    10               1
    11               0
    12               1
    13               2
    14               1

    Training Data Shape: (12, 5)
    Testing Data Shape: (3, 5)
```

EXPLANATION:

The program starts by loading the student health dataset using Pandas. It handles missing values in numerical columns such as height, weight, and BMI by replacing them with their respective mean values. Next, categorical features like medical condition and fitness status are encoded into numerical form using Label Encoding so that they can be used in machine learning models. After encoding, the numerical features are standardized using

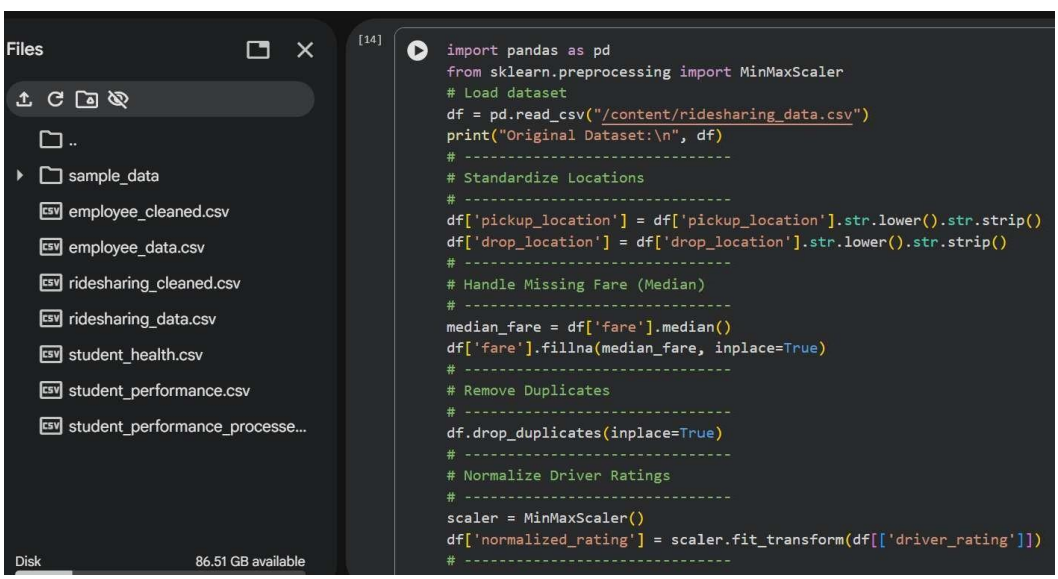
StandardScaler, which transforms the data to have a mean of 0 and a standard deviation of 1 . Finally, the dataset is split into training and testing sets using train_test_split, where 80% of the data is used for training and 20% for testing. This preprocessing ensures that the dataset is clean, consistent, and ready for predictive health analysis.

Task-4: Real-Time Application: Ride-Sharing Data Cleaning

PROMPT:

Write a Python program to clean and preprocess ride-sharing trip data using Pandas and Scikit-learn. The program should standardize pickup and drop locations by converting them into a consistent format, fill missing fare values using the median, remove duplicate ride entries, normalize driver ratings using Min-Max scaling, and generate a summary showing the improvements made to the dataset. The final output should be a clean dataset ready for operational analysis.

CODE:



import pandas as

import

```python
# ------------------------------
# Summary of Improvements
# ------------------------------
print("\nSummary:")
print("Total rows after cleaning:", len(df))
print("Missing values after cleaning:\n", df.isnull().sum())
# ------------------------------
# Final Output
# ------------------------------
print("\nCleaned Dataset:\n", df)
df.to_csv("ridesharing_cleaned.csv", index=False)
```

OUTPUT:

```
Original Dataset:
    ride_id pickup_location drop_location   fare  driver_rating
0         1         Chennai     Bangalore  250.0            4.5
1         2         chennai     Bangalore    NaN            4.7
2         3         CHENNAI     bangalore  300.0            4.5
3         4       Bangalore       Chennai  400.0            3.8
4         5       bangalore       CHENNAI    NaN            4.0
5         6       Hyderabad       Chennai  350.0            4.9
6         7       HYDERABAD       chennai  370.0            4.8
7         8         Chennai     Hyderabad  280.0            4.6
8         9       Bangalore     HYDERABAD    NaN            3.9
9        10       Hyderabad     Bangalore  420.0            4.2
10        3         CHENNAI     bangalore  300.0            4.5
11        5       bangalore       CHENNAI    NaN            4.0

Summary:
Total rows after cleaning: 10
Missing values after cleaning:
 ride_id              0
pickup_location      0
drop_location        0
fare                 0
driver_rating        0
normalized_rating    0
dtype: int64

Cleaned Dataset:
    ride_id pickup_location drop_location   fare  driver_rating  \
0        1         chennai     bangalore  250.0            4.5
```

```
Cleaned Dataset:
    ride_id pickup_location drop_location   fare  driver_rating  \
0        1         chennai     bangalore  250.0            4.5
1        2         chennai     bangalore  325.0            4.7
2        3         chennai     bangalore  300.0            4.5
3        4       bangalore       chennai  400.0            3.8
4        5       bangalore       chennai  325.0            4.0
5        6       hyderabad       chennai  350.0            4.9
6        7       hyderabad       chennai  370.0            4.8
7        8         chennai     hyderabad  280.0            4.6
8        9       bangalore     hyderabad  325.0            3.9
9       10       hyderabad     bangalore  420.0            4.2

   normalized_rating
0           0.636364
1           0.818182
2           0.636364
3           0.000000
4           0.181818
5           1.000000
6           0.909091
7           0.727273
8           0.090909
9           0.363636
```

EXPLANATION:

The program begins by loading the ride-sharing dataset into a Pandas DataFrame. It standardizes the pickup and drop locations by converting all text to lowercase and removing extra spaces to ensure consistency. Missing fare values are handled by replacing them with the median fare, which helps maintain the data distribution without being affected by extreme values. Duplicate ride entries are removed to avoid redundancy and improve data quality. The driver ratings are then normalized using Min-Max scaling, converting them into a range between 0 and 1 for better comparability in analysis. Finally, the program generates a summary showing the number of rows after cleaning and checks for any remaining missing values. The cleaned dataset is displayed and saved as a new CSV file, making it ready for operational analysis.